

Grupo: CCOMP 6-1.1

Fecha: _____

Profesor: Julio Santisteban

Apellidos del alumno _____ Nombre del alumno _____

1. Objetivos de la Evaluación

- Demostrar el dominio de la jerarquía y ciclo de vida de procesos en entornos POSIX/Linux (fork, execv).
- Implementar mecanismos de captura y manejo de señales enviadas desde la interfaz de línea de comandos (signal / sigaction).
- Aplicar dos mecanismos distintos de Comunicación Entre Procesos (IPC): Pipes (Tuberías) y Message Passing (Colas de Mensajes).
- Diseñar diagramas de arquitectura para representar la jerarquía de procesos y el flujo de comunicación interproceso.

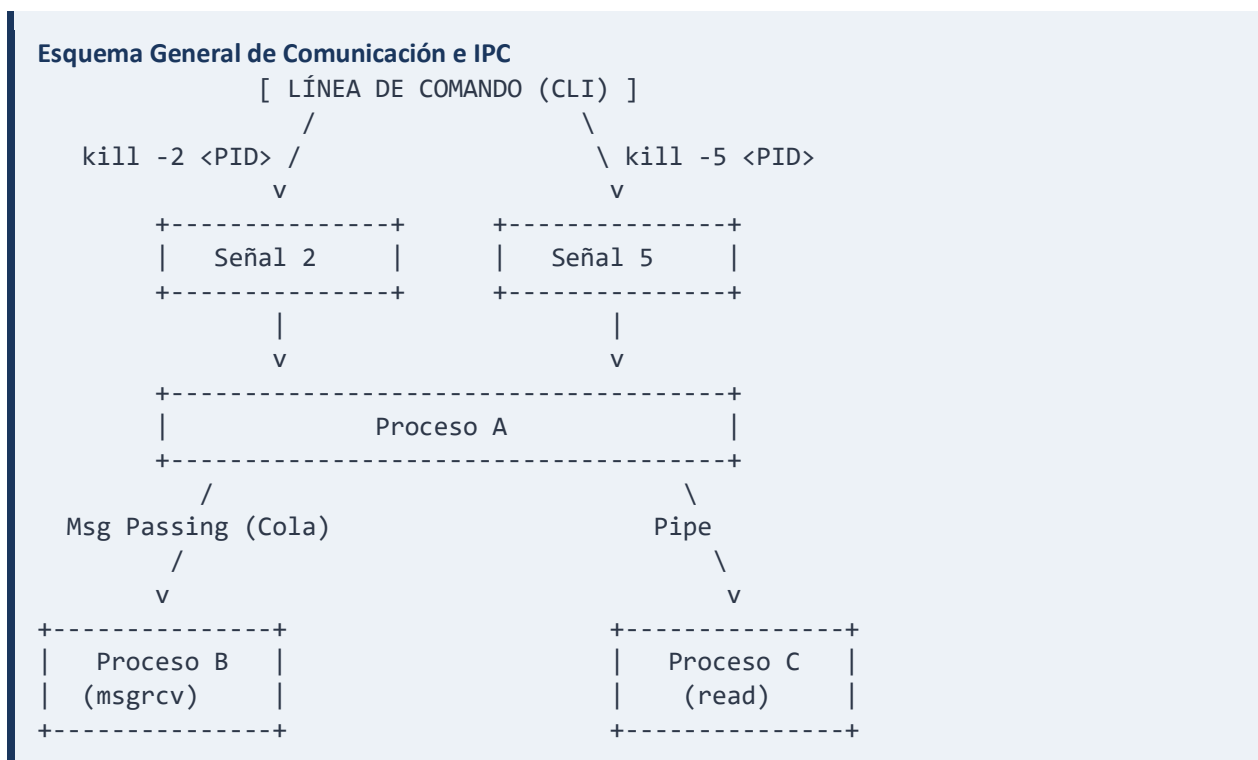
2. Validez de las Señales Asignadas

Antes de implementar la práctica, el estudiante debe tener en cuenta la validación teórica de las señales utilizadas:

- **Señal 2 (SIGINT - Terminal Interrupt):** VÁLIDA. Es la señal estándar que genera el sistema operativo cuando se presiona la combinación de teclas Ctrl+C. Es completamente capturable y permite asociarle un manejador personalizado para ejecutar una acción específica en lugar de finalizar el proceso.
- **Señal 5 (SIGTRAP - Trace/Breakpoint Trap):** VÁLIDA. Es una señal utilizada por mecanismos de depuración (debugging) e interrupción de hardware/software. A diferencia de señales no capturables como SIGKILL (9) o SIGSTOP (19), la señal SIGTRAP sí puede ser capturada y gestionada por un manejador en el código sin provocar el cierre intempestivo del programa.

3. Enunciado y Especificación del Problema

Diseñar e implementar una solución en lenguaje C compuesta por una estructura de tres procesos (A, B y C) que interactúen según la siguiente secuencia de eventos:



Comportamiento Detallado de los Procesos

Proceso A (Proceso Padre Principal):

- Debe inicializar los recursos IPC necesarios (crear la Tubería y la Cola de Mensajes).
- Debe registrar un manejador para escuchar las señales 2 (SIGINT) y 5 (SIGTRAP) enviadas desde la terminal mediante el comando kill.
- Debe mantenerse en espera pasiva de señales.
- Acción ante Señal 2: Al recibir SIGINT, construye un mensaje y lo transmite al Proceso B utilizando el mecanismo de Message Passing (Cola de mensajes POSIX/System V).
- Acción ante Señal 5: Al recibir SIGTRAP, construye un mensaje y lo transmite al Proceso C a través de un Pipe (Tubería sin nombre).

Proceso B (Proceso Hijo):

- Es creado por el Proceso A mediante la llamada al sistema fork().
- Inmediatamente después de nacer, el Proceso B debe clonarse creando un proceso hijo (el 'Clon de B').
- El Proceso B original permanece a la espera de mensajes transmitidos por el Proceso A a través de la Cola de Mensajes.
- Al recibir un mensaje, debe imprimirlo en la consola indicando su identificador de proceso (PID) y el contenido.

Proceso C (Ejecutable Independiente):

- El Clon de B debe reemplazar de inmediato su imagen de memoria mediante la llamada al sistema execv(), transformándose en un ejecutable independiente denominado proceso_c.
- Debe recibir como argumento (vía argv en execv) el descriptor de lectura del Pipe creado inicialmente por el Proceso A.
- Una vez transformado en Proceso C, permanece a la escucha de la tubería.
- Al recibir un mensaje desde el Proceso A, debe imprimirlo en pantalla indicando su PID y el canal de recepción.

